

DSAA2043: Exercise 1

Notes

- This exercise is for practice only and does not count toward the final score.
- Both 0-indexed and 1-indexed array are accepted. However, when using Python, array indices must start from 0.

Problem 1. Insertion Sort is an efficient algorithm for sorting a small number of elements. It works similarly to how people sort a hand of playing cards: each card is picked and inserted into its correct position among the already sorted cards. The algorithm iterates through the array, inserting each element into its appropriate place relative to the previous elements.

Figure 1 and Algorithm 1 demonstrate how Insertion Sort works. The index j represents the current element being inserted into the sorted portion of the array.

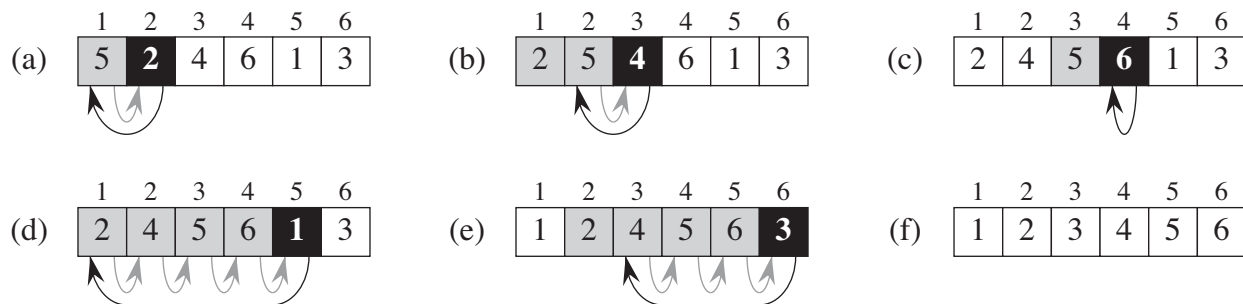


Figure 1: The operation of Insertion-Sort on the array $A = \langle 5, 2, 4, 6, 1, 3 \rangle$.

Algorithm 1 Insertion-Sort(A)

```
1: for  $j = 2$  to  $A.length$  do
2:    $key = A[j]$ 
3:   // Insert  $A[j]$  into the sorted sequence  $A[1 \dots j - 1]$ .
4:    $i = j - 1$ 
5:   while  $i > 0$  and  $A[i] > key$  do
6:      $A[i + 1] = A[i]$ 
7:      $i = i - 1$ 
8:   end while
9:    $A[i + 1] = key$ 
10: end for
```

Answer the following questions:

Q1.1. Using Figure 1 as a reference, illustrate the step-by-step execution of Insertion Sort on the array $A = \langle 31, 41, 59, 26, 41, 58 \rangle$.

Q1.2. Analyze the worst-case time complexity of the algorithm.

Q1.3. Modify Algorithm 1 to sort the array in non-increasing order instead of non-decreasing order.

Answer 1.1: Below is the step-by-step execution of insertion sort on the given array:

Index	0	1	2	3	4	5
Initial	31	41	59	26	41	58
Pass 1	31	41	59	26	41	58
Pass 2	31	41	59	26	41	58
Pass 3	26	31	41	59	41	58
Pass 4	26	31	41	41	59	58
Pass 5	26	31	41	41	58	59

Answer 1.2: In the worst-case scenario (when the array is in descending order but we want to sort it in ascending order), each insertion shifts nearly all of the previously sorted elements. For an array of size n , the total number of comparisons (and shifts in the worst case) is on the order of:

$$1 + 2 + 3 + \cdots + (n - 1) = \frac{n(n - 1)}{2} = O(n^2).$$

Hence, the worst-case time complexity is $O(n^2)$.

Answer 1.3: To sort the array in *non-increasing order* (from largest to smallest) instead of non-decreasing order, we change the comparison in the **while** loop so that we move elements if they are *smaller* than the *key* rather than larger. Specifically, replace

while ($i > 0$ **and** $A[i] > key$)

with

while ($i > 0$ **and** $A[i] < key$).

Below is the modified pseudocode:

Algorithm 2 Insertion-Sort-Non-Increasing(A)

```

1: for  $j = 2$  to  $A.length$  do
2:    $key = A[j]$ 
3:   // Insert  $A[j]$  into the sorted sequence  $A[1 \dots j - 1]$  in non-increasing order
4:    $i = j - 1$ 
5:   while  $i > 0$  and  $A[i] < key$  do
6:      $A[i + 1] = A[i]$ 
7:      $i = i - 1$ 
8:   end while
9:    $A[i + 1] = key$ 
10: end for

```

Problem 2. Selection sort is a simple sorting algorithm that sorts an array by repeatedly selecting the smallest remaining element and placing it in its correct position. Given an array A of n numbers, the algorithm works as follows:

1. Find the smallest element in A and swap it with the first element, $A[1]$.
2. Find the second smallest element in the remaining unsorted part of A and swap it with $A[2]$.
3. Repeat this process for the first $n - 1$ elements of A .

Q2.1. Write the pseudocode or Python implementation of the selection sort algorithm.

Q2.2. Perform a step-by-step computational cost analysis of the algorithm. Define c_i as the *cost* of step i , and determine how many *times* each step is executed. Use this analysis to derive the total running time $T(n)$.

Q2.3. Express the best-case and worst-case time complexities of selection sort using Θ -notation.

Answer 2.1:

Pseudocode (1-based indexing for clarity):

Algorithm 3 Selection-Sort(A)

```

1: for  $i = 1$  to  $A.length$  do
2:    $min\_index \leftarrow i$                                  $\triangleright$  Assume the smallest element is at position  $i$ 
3:   for  $j = i + 1$  to  $n$  do
4:     if  $A[j] < A[min\_index]$  then
5:        $min\_index \leftarrow j$ 
6:     end if
7:   end for
8:   Swap  $A[i]$  and  $A[min\_index]$ 
9: end for

```

Python implementation (0-based indexing):

```

def selection_sort(A):
    n = len(A)
    for i in range(n - 1):
        min_index = i
        for j in range(i+1, n):
            if A[j] < A[min_index]:
                min_index = j

        # Swap A[i] with A[min_index]
        A[i], A[min_index] = A[min_index], A[i]

```

Answer 2.2: Let $n = A.length$. For each $i = 1, \dots, n - 1$, let t_i denote the number of times the condition in line 4 is true. We use a constant c_i to represent the cost of each line and record how often each line is executed.

Selection-Sort(A)		
Line	Cost	Times
for $i = 1$ to $n = A.length$	c_1	n
$min_index \leftarrow i$	c_2	$n - 1$
for $j = i + 1$ to n	c_4	$\sum_{i=1}^{n-1} (n - i)$
if $A[j] < A[min_index]$	c_5	$\sum_{i=1}^{n-1} (n - i - 1)$
$min_index \leftarrow j$	c_6	$\sum_{i=1}^{n-1} t_i$
Swap $A[i]$ and $A[min_index]$	c_8	$n - 1$

Total running time $T(n)$:

$$\begin{aligned}
T(n) = & c_1 n + c_2 (n - 1) + c_4 \sum_{i=1}^{n-1} (n - i - 1) + c_5 \sum_{i=1}^{n-1} (n - i) \\
& + c_6 \sum_{i=1}^{n-1} t_i + c_8 (n - 1).
\end{aligned}$$

Answer 2.3: Since the inner loop executes $\sum_{i=1}^{n-1} (n - i) = \frac{n(n-1)}{2}$ times, the comparisons and possible updates to `min_index` occur $O(n^2)$ times overall. Consequently, the dominant term is $O(n^2)$, leading to

$$T(n) = \Theta(n^2).$$

Regardless of whether the array is already sorted or in reverse order, SELECTION-SORT performs the same number of comparisons. Hence, the **best-case** and **worst-case** running times are both $\Theta(n^2)$.

Problem 3. Binary search is an efficient algorithm for finding a target value in a sorted array by repeatedly dividing the search range in half. Given a sorted array A of size n and a target value x , answer the following questions:

Q3.1. Provide a pseudocode or Python implementation of the binary search algorithm.

Q3.2. Prove that the worst-case time complexity of binary search is $O(\log n)$.

Q3.3. Perform a step-by-step execution of the algorithm on the array $A = \langle 2, 3, 5, 7, 11, 13, 17, 19 \rangle$ when searching for $x = 17$.

Answer 3.1:

Pseudocode 1:

Algorithm 4 Binary-Search(A, x)

```

1: low  $\leftarrow$  1
2: high  $\leftarrow$  A.length
3: while low  $\leq$  high do
4:   mid  $\leftarrow$   $\lfloor (\text{low} + \text{high})/2 \rfloor$ 
5:   if A[mid] =  $x$  then
6:     return mid
7:   else if A[mid] >  $x$  then
8:     high  $\leftarrow$  mid - 1
9:   else
10:    low  $\leftarrow$  mid + 1
11:   end if
12: end while
13: return -1

```

Pseudocode 2:

Algorithm 5 Binary Search (Recursive Implementation)

```

1: function BINARYSEARCH(A, low, high, x)
2:   if low > high then
3:     return -1
4:   end if
5:   mid  $\leftarrow$   $\lfloor (\text{low} + \text{high})/2 \rfloor$ 
6:   if A[mid] ==  $x$  then
7:     return mid
8:   else if A[mid] >  $x$  then
9:     return BinarySearch(A, low, mid-1, x)
10:  else
11:    return BinarySearch(A, mid+1, high, x)
12:  end if
13: end function

```

Python implementation:

```
def binary_search(A, x):
```

```

low = 0
high = len(A) - 1

while low <= high:
    mid = (low + high) // 2
    if A[mid] == x:
        return mid
    elif A[mid] > x:
        high = mid - 1
    else:
        low = mid + 1

return -1 # x not found

```

Answer 3.2: Each comparison in the binary search algorithm cuts the search range in half.

- Initially, the search range has size n .
- After the first comparison, the search range is at most $\frac{n}{2}$ in size.
- After the second comparison, the search range is at most $\frac{n}{4}$ in size.
- After k comparisons, the search range is at most $\frac{n}{2^k}$ in size.
- The algorithm terminates when the size of the search range becomes 1 (or 0), which implies:

$$\frac{n}{2^k} \leq 1 \implies n \leq 2^k \implies k \geq \log_2(n).$$

- Therefore, the maximum number of comparisons (and thus the number of loop iterations) is on the order of $\log n$.

Hence, the worst-case time complexity of binary search is $O(\log n)$.

Answer 3.3:

Step	Low	High	Mid	$A[\text{mid}]$
1	0	7	$\lfloor (0 + 7)/2 \rfloor = 3$	$A[3] = 7$
2	4	7	$\lfloor (4 + 7)/2 \rfloor = 5$	$A[5] = 13$
3	6	7	$\lfloor (6 + 7)/2 \rfloor = 6$	$A[6] = 17$

Problem 4. Answer the following questions:

A	B	O	Ω	Θ
$5n$	n	✓	✓	✓
5	n	✓		
n^2	$2^{10}n$			
$(7/8)^n$	$(8/7)^n$			
8^n	4^n			
n^5	$32^{\log n}$			
n^{2025}	2025^n			
$n^{0.8}$	$(0.8)^n$			
$n^{\log 3}$	$3^{\log n}$			
$\log(n!)$	$\log(n^n)$			
$n^{1/\log n}$	1			
$\log^7 n$	$n^{0.7}$			
$\log \sqrt{n}$	$\sqrt[4]{n}$			
$\log(\sqrt{n})$	$\sqrt{\log n}$			

Q4.1. For each row i in the table above, determine whether A_i belongs to $O(B_i)$, $\Omega(B_i)$, or $\Theta(B_i)$. Mark all applicable classifications. No explanation is required. The first two rows are provided as examples.

Q4.2. Let $f(n)$, $g(n)$, and $h(n)$ be functions of integer n . Prove: if $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

Q4.3. Prove $(2n + 2)^3 = O(n^3)$.

Q4.4. Prove $\log_2(n!) = \Theta(n \log n)$.

Answer 4.1: See Table 1.

(3) $A = n^2$ vs. $B = 2^{10}n$.

As $n \rightarrow \infty$, n^2 grows faster than any constant multiple of n . Hence $n^2 \notin O(n)$, but $n^2 \in \Omega(n)$. Not Θ because they do not grow at the same rate.

(4) $A = (\frac{7}{8})^n$ vs. $B = (\frac{8}{7})^n$.

$(\frac{7}{8})^n$ decays exponentially, whereas $(\frac{8}{7})^n$ grows exponentially. One can show $(\frac{7}{8})^n \in O(\frac{8}{7})^n$ (it is much smaller), but not in Ω or Θ .

(5) $A = 8^n$ vs. $B = 4^n$.

$8^n = (2^3)^n = 2^{3n}$ and $4^n = 2^{2n}$. Thus $8^n/4^n = 2^n \rightarrow \infty$, meaning 8^n grows faster, so $8^n \in \Omega(4^n)$ but not in O or Θ .

(6) $A = n^5$ vs. $B = 32^{\log n}$.

Note $32^{\log n} = (2^5)^{\log n} = 2^{5 \log n} = (2^{\log n})^5 = n^5$. They are exactly the same function, implying $n^5 \in O(n^5)$, $\Omega(n^5)$, and $\Theta(n^5)$.

A	B	O	Ω	Θ
$5n$	n	✓	✓	✓
5	n	✓		
n^2	$2^{10}n$		✓	
$(\frac{7}{8})^n$	$(\frac{8}{7})^n$	✓		
8^n	4^n		✓	
n^5	$32^{\log n}$	✓	✓	✓
n^{2025}	2025^n	✓		
$n^{0.8}$	$(0.8)^n$		✓	
$n^{\log 3}$	$3^{\log n}$	✓	✓	✓
$\log(n!)$	$\log(n^n)$	✓	✓	✓
$n^{1/\log n}$	1	✓	✓	✓
$\log^7 n$	$n^{0.7}$	✓		
$\log(\sqrt{n})$	$\sqrt[4]{n}$	✓		
$\log(\sqrt{n})$	$\sqrt{\log n}$		✓	

Table 1: Big-O, Omega, and Theta classification.

- (7) $A = n^{2025}$ vs. $B = 2025^n$.

A large polynomial vs. an exponential. The exponential 2025^n dominates any polynomial, so $n^{2025} \in O(2025^n)$. Not in Ω or Θ .

- (8) $A = n^{0.8}$ vs. $B = (0.8)^n$.

$(0.8)^n$ decays to 0, whereas $n^{0.8}$ grows unbounded. Hence $n^{0.8}$ is never $O((0.8)^n)$, but it does exceed it eventually, so $n^{0.8} \in \Omega((0.8)^n)$.

- (9) $A = n^{\log 3}$ vs. $B = 3^{\log n}$.

We can rewrite $3^{\log n} = e^{(\log 3)(\log n)}$ and $n^{\log 3} = e^{(\log 3)(\log n)}$. They are identical up to the same exponent. Thus all three: O, Ω, Θ hold.

- (10) $A = \log(n!)$ vs. $B = \log(n^n)$.

By Stirling's approximation, $\log(n!) \approx n \log n$, and $\log(n^n) = n \log n$. They grow at the same rate, so $\log(n!) \in \Theta(n \log n)$. Hence $\log(n!) \in O(\log(n^n))$, $\Omega(\log(n^n))$, and $\Theta(\log(n^n))$.

- (11) $A = n^{1/\log n}$ vs. $B = 1$.

Interpreting $\log$ as the natural log, $n^{1/\log n} = e^{(\log n)(1/\log n)} = e$, a constant. Any constant function is both $O(1)$ and $\Omega(1)$, hence $\Theta(1)$.

- (12) $A = \log^7 n$ vs. $B = n^{0.7}$.

Polynomial $n^{0.7}$ outgrows any polylog $\log^7 n$. Thus $\log^7 n \in O(n^{0.7})$, but not in Ω or Θ .

- (13) $A = \log(\sqrt{n})$ vs. $B = \sqrt[4]{n}$.

We have $\log(\sqrt{n}) = \frac{1}{2} \log n$ and $\sqrt[4]{n} = n^{1/4}$. Since $\log n$ grows slower than $n^{1/4}$, $\frac{1}{2} \log n \in O(n^{1/4})$. Not in Ω or Θ .

(14) $A = \log(\sqrt{n}) = \frac{1}{2} \log n$ vs. $B = \sqrt{\log n}$.

Comparing $\log n$ vs. $(\log n)^{1/2}$: as n grows, $\log n$ is larger than $(\log n)^{1/2}$. Hence $\frac{1}{2} \log n \notin O(\sqrt{\log n})$. Instead, $\frac{1}{2} \log n \in \Omega(\sqrt{\log n})$. Not Θ , since they differ more than a constant factor.

Answer 4.2:

Since $f(n) = O(g(n))$, there exist constants c_1, c_2 such that for all $n \geq c_2$, it holds that

$$f(n) \leq c_1 g(n).$$

Similarly, since $g(n) = O(h(n))$, there exist constants c'_1, c'_2 such that for all $n \geq c'_2$, it holds that

$$g(n) \leq c'_1 h(n).$$

Set $c''_1 = c_1 c'_1$ and $c''_2 = \max\{c_2, c'_2\}$. From the above, we know that for all $n \geq c''_2$, it holds that

$$f(n) \leq c_1 g(n) \leq c_1 c'_1 h(n) = c''_1 h(n).$$

Therefore, $f(n) = O(h(n))$.

Answer 4.3: To show that $(2n+2)^3 \in O(n^3)$, we need to find positive constants C and n_0 such that for all $n \geq n_0$, $(2n+2)^3 \leq C n^3$. First, expand $(2n+2)^3$: $(2n+2)^3 = 8n^3 + 24n^2 + 24n + 8$. We want to show that $8n^3 + 24n^2 + 24n + 8 \leq C n^3$ for some constant C and sufficiently large n . Observe:

$$8n^3 + 24n^2 + 24n + 8 \leq 8n^3 + 24n^3 + 24n^3 + 8n^3 = 64n^3,$$

provided that $n \geq 1$ (since $n^2 \leq n^3$ and $n \leq n^3$ for $n \geq 1$). Thus, for all $n \geq 1$:

$$(2n+2)^3 = 8n^3 + 24n^2 + 24n + 8 \leq 64n^3.$$

Hence, if we choose $C = 64$ and $n_0 = 1$, we have

$$(2n+2)^3 \leq C n^3 \quad \text{for all } n \geq n_0.$$

By definition of Big-O notation, this proves $(2n+2)^3 \in O(n^3)$.

Answer 4.4: Let us first prove $\log_2(n!) = O(n \log n)$:

$$\begin{aligned} \log_2(n!) &= \log_2 \left(\prod_{i=1}^n i \right) \\ &\leq \log_2 n^n \\ &= n \log_2 n \\ &= O(n \log n). \end{aligned}$$

Now we prove $\log_2(n!) = \Omega(n \log n)$:

$$\begin{aligned} \log_2(n!) &= \log_2 \left(\prod_{i=1}^n i \right) \\ &\geq \log_2 \left(\prod_{i=n/2}^n i \right) \\ &\geq \log_2 (n/2)^{n/2} \\ &= (n/2) \log_2(n/2) \\ &= \Omega(n \log n). \end{aligned}$$

This completes the proof.

Problem 5. A financial analyst is monitoring stock prices over n days, where the array A represents the stock price on each day, with $A[i]$ denoting the price on day i . The analyst seeks to determine the maximum profit that can be obtained by buying and selling the stock exactly once.

Answer the following questions:

Q5.1. Provide pseudocode or Python code for an algorithm that computes the maximum possible profit from a given array A of length n .

Input: A sequence of n numbers $A = \langle a_1, a_2, \dots, a_n \rangle$.

Output: The maximum possible profit v , defined as $v = A[j] - A[k]$, where $1 \leq k < j \leq n$.

Q5.2. Analyze the time and space complexity of your algorithm.

Answer 5.1:

Pseudocode:

Algorithm 6 Max Profit in Stock Prices

```

1: function MAXPROFIT( $A, n$ )
2:    $\text{minPrice} \leftarrow A[1]$ 
3:    $\text{maxProfit} \leftarrow 0$ 
4:   for  $i = 1$  to  $A.\text{length}$  do
5:      $\text{profit} \leftarrow A[i] - \text{minPrice}$ 
6:      $\text{maxProfit} \leftarrow \max(\text{maxProfit}, \text{profit})$ 
7:      $\text{minPrice} \leftarrow \min(\text{minPrice}, A[i])$ 
8:   end for
9:   return  $\text{maxProfit}$ 
10: end function

```

Python Implementation:

```

def max_profit(A):
    n = len(A)
    if n < 2:
        return 0 # No profit possible with fewer than 2 days
    min_price = A[0]
    max_profit = 0
    for i in range(1, n):
        current_price = A[i]
        profit = current_price - min_price
        if profit > max_profit:
            max_profit = profit
        if current_price < min_price:
            min_price = current_price
    return max_profit

```

Answer 5.2:

- Time Complexity: Since we only iterate through the array once, each step involves a constant amount of work (updating minPrice and maxProfit). Therefore, the time complexity is $O(n)$.
- Space Complexity: We only use a constant amount of additional space (for minPrice , maxProfit , and a few temporary variables). Hence, the space complexity is $O(1)$.